

de-ai-agent

Interview Preparation Guide

Multi-agent data engineering + AI query pipeline — FIFA World Cup 2026 dataset

30-Second Summary

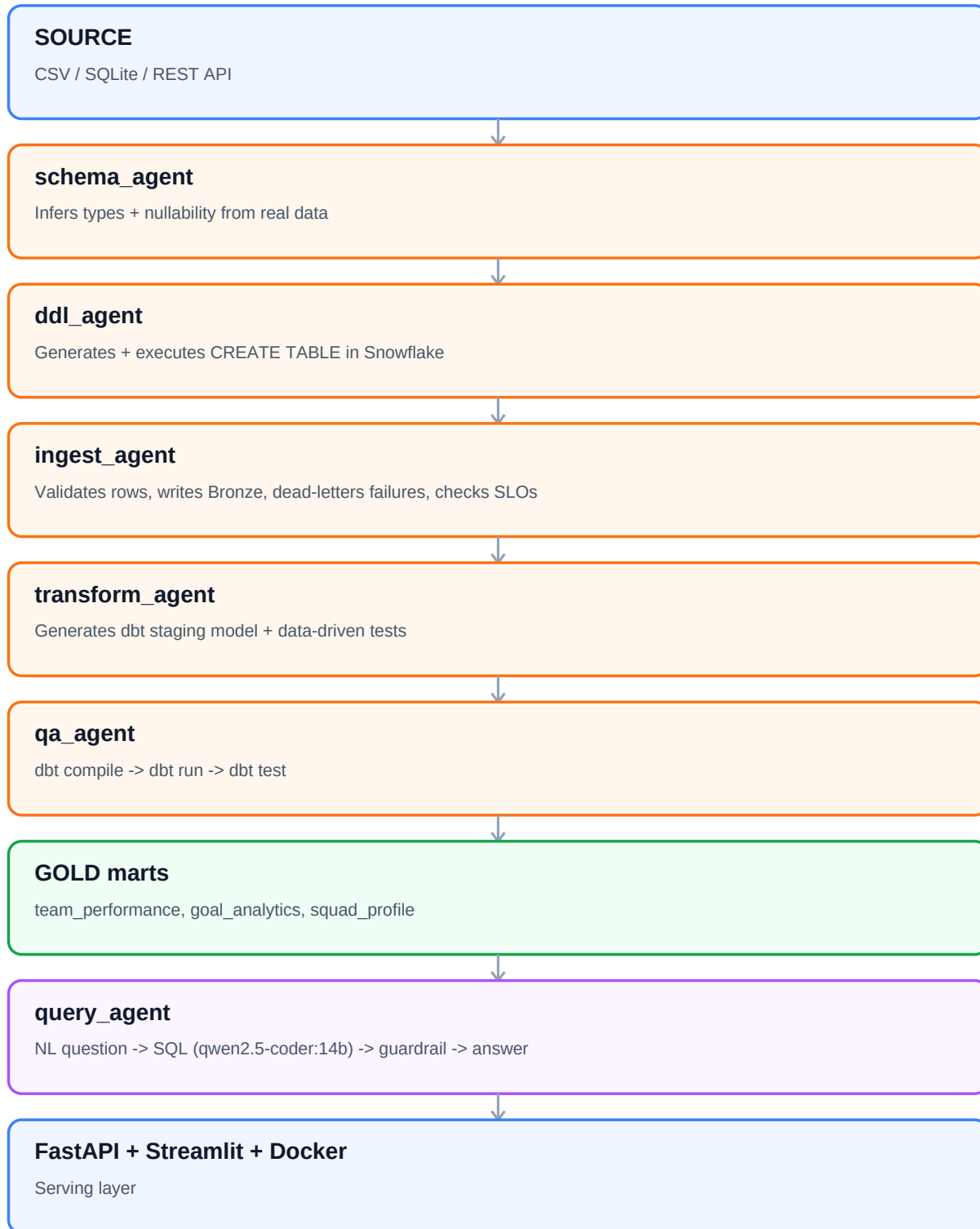
A multi-agent LangGraph pipeline that ingests FIFA World Cup 2026 data from 6 heterogeneous sources (CSV, SQLite, REST API) into Snowflake, transforms it with dbt into Gold-layer analytics marts, and exposes it through a local-LLM (qwen2.5-coder:14b) natural-language query agent with guardrails, an evaluation suite, a FastAPI service, Docker containerization, and a Streamlit UI.

What This Demonstrates

- **Multi-agent orchestration** (LangGraph): 5 agents run as a deterministic state graph per source, with retry logic and error routing.
- **Heterogeneous ingestion**: CSV, SQLite, and a live REST API in one pipeline — three real-world source patterns.
- **Data-driven quality profiling**: a single-pass column profiler computes null rate, uniqueness, and value ranges from real sampled data and auto-generates the correct dbt tests — not hardcoded per source.
- **Governance**: automatic PII detection, Snowflake query tagging for cost attribution, dead-letter routing, SLO breach detection.
- **Real data only**: all 6 sources use verifiable, real data sourced from openfootball's public API — synthetic/fabricated data was explicitly rejected twice during development.
- **AI engineering layer**: local LLM query agent with SQL guardrails, conversational routing (distinguishes data questions from greetings/meta-questions), session memory (resolves follow-up questions like "what about their goals"), an eval suite, Docker deployment, and a Streamlit front end.

Architecture

One LangGraph run per source. Agents execute in strict sequence; a deterministic supervisor (not an LLM) reads state after each node and decides the next hop. Retries live inside each agent via tenacity — the graph itself stays acyclic.



The Six Sources

Source	Type	Rows	Notes
historical_results	CSV	49,481	International match results, 1872-present
historical_goals	CSV	47,575	Goalscorers linked to results
historical_shootouts	CSV	680	Penalty shootout outcomes
worldcup_api	REST API	412	Live WC 2026 fixtures/scores, 2 tables
national_teams	SQLite	48	Real WC 2026 qualified teams + groups
player_profiles	SQLite	1,248	Real WC 2026 full squad rosters

Stack

- **Orchestration:** LangGraph (multi-agent pipeline) + Prefect (designed, not yet deployed to a live server)
- **Warehouse:** Snowflake — Bronze (raw), Silver (staging, 1:1 renames), Gold (marts, real joins/aggregations)
- **Transformation:** dbt + dbt_expectations, dbt_date packages
- **Ingestion:** snowflake-connector-python, httpx, sqlite3
- **LLM:** local Ollama, qwen2.5-coder:14b (migrated from llama3.1:8b) — zero marginal cost, code-specialized model
- **Serving:** FastAPI (/v1/nl-to-sql), Streamlit UI, Docker containerized

Real Bugs Found and Fixed

This is the most valuable interview material in this document — it demonstrates real debugging under pressure, not just following a tutorial. Pick 2-3 of these to walk through in detail if asked "tell me about a bug you fixed."

Settings field mismatches

Four files (orchestrator, ingestor, snowflake_writer, contracts) called a `get_settings()` function that was never defined — only a settings object existed. Found via systematic diff of every settings.X reference in the codebase against every field actually defined in `config/settings.py`.

Dead-letter and watermark schema drift

The Python code writing to `BRONZE.DEAD_LETTER` and `BRONZE.INGESTION_WATERMARKS` used column names that didn't match the tables' real schema — built by an earlier iteration of the code with different column names. Fixed by querying `DESCRIBE TABLE` and aligning the `INSERT/MERGE` statements to reality.

Snowflake PARSE_JSON in batched VALUES

Snowflake rejects function calls like `PARSE_JSON()` inside a batched `INSERT...VALUES` clause — only allowed in a `SELECT` list. Fixed by switching to `INSERT...SELECT` with `PARSE_JSON` in the select list.

Silent dbt subprocess failures

`qa_agent`'s dbt compile/test subprocess calls used the wrong `--profiles-dir`, failing with an empty `stderr` that gave no diagnostic signal. Traced by temporarily adding a full Python traceback dump before the error was swallowed.

Missing dbt run before dbt test

`qa_agent` called dbt test without first calling dbt run — every test failed because the model was never materialized in Snowflake. Simple but easy to miss when compile succeeds silently.

PII substring false positive

PII detection used substring matching, so "tournament" was flagged as PII because it contains "name" (tour-NAME-nt). Fixed with word-boundary regex matching.

Encoding bug on non-ASCII data

Files were opened without `encoding="utf-8"`, causing a `UnicodeDecodeError` on team names like "Curacao". A one-line fix, but the kind of bug that only surfaces with real international data.

Snowflake least-privilege grants

`DE_TRANSFORM_ROLE` needed explicit one-time `GRANT CREATE SCHEMA`, and later `GRANT CREATE VIEW`, before it could create new object types — a recurring theme of least-privilege roles needing explicit per-object-type grants.

Sample-based accepted_values test failed at scale

An auto-generated dbt `accepted_values` test was built from a 500-row sample of the tournament column, but the full 49k-row dataset had more real tournament values than the sample saw — producing false test failures. Resolved by surfacing categorical candidates as metadata for review instead of auto-enforcing a hard constraint from a sample.

dbt schema-name doubling (SILVER_SILVER)

A custom `+schema` config in `dbt_project.yml` combined with dbt's default naming macro produced a doubled schema name (`SILVER_SILVER`) after a manual schema rename in the Snowflake console. Fixed with a `generate_schema_name` macro override that uses the custom schema name directly.

Gold-layer routing misconfiguration

`dbt_project.yml` had a config key named `gold:` but the actual folder was `models/marts/` — dbt requires an exact folder-name match, so the config silently never applied and `marts` defaulted to the Silver schema. Fixed by renaming the config key to `marts:`.

LLM-generated trailing semicolons

The local LLM occasionally generated valid standalone SQL ending in a semicolon, which breaks inside dbt's templated model wrapper. Fixed by stripping trailing semicolons in code before writing any generated model to disk — applies to all future sources automatically.

TRY_CAST type mismatch

Snowflake rejected `TRY_CAST(already_numeric_column AS NUMBER)` when the source type was `FLOAT` — a real Snowflake type-system quirk. Fixed using `:: cast` syntax instead.

Recurring stale-environment-variable failures

dbt's subprocess calls read Snowflake credentials from real shell environment variables, not from `.env` directly — every new terminal session needed ``set -a; source .env; set +a`` run manually before dbt would connect, and this was missed multiple times across the build, each time producing the same empty-stderr failure signature. The real lesson: a recurring failure with an identical signature should be fixed at the code level (e.g. loading env vars automatically in Python), not re-diagnosed from scratch each time it appears.

Key Design Decisions

Full detail lives in docs/adr/DECISIONS.md in the repo. Summarized here for quick recall.

Staging strictly 1:1; joins only in marts

Matches dbt Labs' standard convention. Keeps the foundational Bronze->Silver layer stable and low-risk to depend on; all business logic and joins live in a separate, independently-evolvable Gold layer.

LangGraph over Airflow

Airflow's scheduler + metadata DB infrastructure wasn't justified for a single-node portfolio project. LangGraph gives deterministic routing and retry semantics with far less operational weight. Airflow would be reconsidered at real multi-node production scale.

Dynamic DDL from inferred schema

Zero hand-written DDL per source — schema_agent infers types from real data, ddl_agent generates CREATE TABLE dynamically. Adding a new source requires zero SQL work, just a registry entry and a fetch function.

Data-driven column profiling over hardcoded rules

A single-pass profiler computes null rate, uniqueness, and value ranges from real data and generates the appropriate dbt tests automatically — the same pattern used by tools like Great Expectations, implemented at small scale.

dbt_expectations over hand-rolled tests

Adopted the maintained, widely-used industry package for range/distribution tests instead of continuing to hand-write one-off singular SQL tests per numeric column — a deliberate signal that the tradeoff was considered, not defaulted to build-it-myself.

Real data only, twice rejected synthetic fallback

When national_teams and player_profiles initially had no data source, synthetic/generated placeholder data was considered and explicitly rejected. Real WC 2026 team and full squad roster data was sourced from openfootball's public-domain API instead — the same free source already used for match results.

Local LLM over API model

qwen2.5-coder:14b via Ollama gives zero marginal cost per query and is code-specialized, outperforming a general-purpose model at SQL generation, while fitting comfortably in 24GB RAM. A paid API model (e.g. Claude) would be the natural upgrade path if token cost were not a constraint.

Deterministic SQL guardrails, not just prompt instructions

Every LLM-generated query passes through a keyword + statement-start validator (sql_guardrails.py) before execution, regardless of what the model produces. This is a code-level enforcement, not a prompt the model could ignore.

Prefect for orchestration, deferred to Prefect Cloud in production

The retry/task-isolation flow is written, but a self-hosted Prefect server was not stood up given time constraints. Real production deployment would use Prefect Cloud's free tier rather than self-hosting, matching how most small teams actually run managed orchestration.

AI Engineer Competency Mapping

Assessed against the core responsibilities of an AI Engineer role: system integration, orchestration, evaluation, and production reliability.

Competency	Status	Evidence
Model selection & inference setup	Done	qwen2.5-coder:14b via Ollama, chosen over llama3.1:8b for code-specialized SQL generation
Agentic workflow design & tool binding	Done	LangGraph pipeline: agents call Snowflake, run dbt tasks, query schemas dynamically
Production guardrails & error handling	Done	sql_guardrails.py blocks DROP/DELETE/UPDATE/etc.; retry loop feeds Snowflake connector
Evaluation & benchmarking	Done	10 golden NL/SQL query pairs, tracks execution success rate, correct-table selection, a
API deployment & serving	Done	FastAPI /v1/nl-to-sql endpoint, containerized with Docker, verified working end-to-end in
Observability & tracing	Not done	structlog logging + Snowflake query tagging exist; no LangSmith/Phoenix step-level tra

10-Point Agent Engineering Framework Self-Assessment

Solid	Orchestration patterns, Context engineering
Partial	Harness engineering, Loop engineering, Tool design, Guardrails/permissions, Observability/tracing
Missing	Memory architecture (partially closed by session history added late in build), Evals for agents (closed — see above)

Governance

Bronze retention policy

Raw Bronze data: 90-day retention (reproducible from source, not the system of record). Silver/Gold: indefinite (cheaper to keep than reconstruct).

Schema-change handling

Known gap: a source's schema changing today would silently drop new columns from inserts rather than alerting. Intended design: diff inferred schema against the live table before ingest and halt for human review.

Role-to-access map

DE_INGEST_ROLE: Bronze insert/select only. DE_TRANSFORM_ROLE: create schema/view on Silver+Gold, select on Bronze. No agent role has DROP/DELETE/cross-schema write access. query_agent's SQL is additionally constrained in code to SELECT/WITH only.

PII handling

Whole-word keyword matching (name, scorer, player, person, email, phone, address) auto-tags columns in dbt schema.yml. Known limitation: value-level PII (e.g. unusually-named columns) would be missed — a production system would supplement with value-pattern scanning.

Human-in-the-loop boundaries

Never auto-executed: any DDL/DML beyond SELECT, role/permission grants, schema changes to existing tables, dead-letter reprocessing. All require explicit human review.

Customer-Facing AI Governance (scoped for the query agent, not yet built)

- Per-user access control (row/column-level security, not one shared role)
- Query scope enforcement restricted to Gold-layer tables only, enforced at the database permission level
- Rate limiting per user to prevent runaway query costs
- Output sanitization before returning results to a customer
- Audit logging tied to the actual user, not just the agent
- Input validation on the natural-language question itself (length limits, injection-pattern blocking)

Known Limitations (Say These Proactively)

Naming these unprompted in an interview is a stronger signal than waiting to be asked — it shows self-awareness of the system's real boundaries, not just what works.

- **Query agent scope handling is inconsistent:** correctly refuses genuinely unanswerable questions (e.g. "who won the 2026 World Cup" — the tournament isn't finished in the dataset), but a live INFORMATION_SCHEMA categorical-value lookup added late in the build occasionally degrades answer quality on category-filtered questions. Needs hardening: fallback to cached/static values, or better prompt grounding with few-shot examples.
- No automated schema-diff detection — a source's schema changing would currently insert silently with dropped columns rather than alerting.
- PII detection is keyword-based on column names only, not value-level content scanning.
- Prefect orchestration flow is written (retry config, per-source task isolation) but not yet deployed against a live server — local self-hosted setup hit connection issues; production plan is Prefect Cloud.
- No Docker networking tested beyond local host.docker.internal — multi-container/cloud deployment networking is unverified.
- Session memory (conversational follow-up resolution) was implemented late in the build and not extensively tested under varied conversation patterns.
- No observability/tracing tool (LangSmith or Phoenix) wired in yet — only structlog logging and Snowflake query tags exist today.
- No formal data classification system beyond PII tagging.
- No row/column-level access control for a future multi-tenant scenario.

Likely Interview Questions

Q: Walk me through this project end to end.

A: Start with the 30-second summary, then walk the pipeline diagram: source -> schema inference -> DDL -> ingestion -> dbt transform -> QA -> Gold marts -> query agent -> serving layer. Emphasize that every step is dynamic (no hardcoded schemas) and every step has a real governance/quality gate.

Q: Tell me about a bug you found and fixed.

A: Pick 2-3 from the Real Bugs section — the dbt schema-doubling bug and the Gold-routing misconfiguration are strong choices since they show root-cause diagnosis across a full stack (dbt config -> Snowflake naming -> code sync), not just a one-line fix.

Q: Why LangGraph instead of a plain Python script with try/except?

A: Routing logic is centralized and auditable independent of agent internals; adding a new agent node or changing routing doesn't require touching existing agent logic. The graph topology is visible and testable separately from what each node does.

Q: How do you handle a source's schema changing?

A: Honest answer: currently a known gap. Walk through the intended design (diff inferred schema against the live table before ingest, halt and alert rather than silently drop columns) to show you understand the problem even though it isn't built.

Q: Why a local LLM instead of an API model like GPT-4 or Claude?

A: Zero marginal cost during heavy iterative development; qwen2.5-coder:14b is code-specialized and outperformed a general-purpose local model at SQL generation while fitting comfortably in available RAM. A paid API model would be the natural next step if quality on harder questions became the priority over cost.

Q: How do you prevent the LLM from generating destructive SQL?

A: Deterministic keyword + statement-start validation in `sql_guardrails.py` runs on every generated query before execution, independent of what the model actually produces or how it was prompted — not just a prompt instruction the model could ignore.

Q: What would you do differently at 10x data volume?

A: Move Bronze ingestion from `executemany()` batches to `COPY INTO` with staged files (already flagged as a threshold in the code's own docstring), move orchestration to Prefect Cloud instead of manual runs, and add schema-diff alerting before it becomes a real production risk.

Q: What's the weakest part of this project right now?

A: The query agent's handling of ambiguous or out-of-scope questions is inconsistent — a live categorical-value lookup added late in the build sometimes degrades answer quality. This is an honest, current limitation, not something to hide.

Q: How did you validate the data quality logic actually works?

A: A single-pass profiler computes real statistics (null rate, uniqueness, value ranges) from sampled data and auto-generates dbt tests from those facts, using the `dbt_expectations` package. Validated end-to-end: a real bug (shootouts data being wrongly rejected on a legitimately-sparse column) was found and fixed using this exact system, and the fix generalized correctly to a completely different source type (an API-driven source) without further changes.

Process & Pacing Lessons (Honest Self-Review)

Worth naming directly if asked "what would you do differently" — these are about how the work was paced, not the technical decisions themselves.

- **Scope creep without checkpoints:** started the session with a narrower goal and kept adding capability (query agent, guardrails, evals, Docker, Streamlit, session memory) without pausing to ask whether finishing what was half-built mattered more than starting the next thing.
- **"Compiles" was sometimes treated as "done":** several fixes were confirmed as syntactically valid and assumed correct without a real end-to-end rerun, which meant some bugs resurfaced later under different conditions.
- **Live infrastructure changes (Snowflake schema renames) made close to a deadline, without first mapping every code reference to the old name** — this produced a cascading, time-costly failure that a controlled, version-tracked migration (or simply not renaming under time pressure) would have avoided.
- **A recurring bug (stale env vars) was re-diagnosed from scratch multiple times instead of being fixed permanently at the code level the first time it appeared.**

Next Steps / Roadmap

What's scoped for future sessions, in priority order:

- Harden query agent scope handling — few-shot examples in the prompt, explicit statement of what's out of scope, stabilize or remove the live categorical-value lookup
- Chart generation refinement — Plotly + fixed styling template (built, working, could be extended with more chart types)
- Deploy Prefect flow to Prefect Cloud for real scheduled orchestration
- Wire in LangSmith or Phoenix for step-level observability and tracing
- Automated schema-diff detection with halt-and-alert behavior
- Formal memory architecture review — session history was added late, needs broader testing
- Customer-facing governance layer (per-user access, rate limiting, audit logging) if/when this moves toward multi-user
- Case study write-up for portfolio site, structured as Problem -> Architecture -> Key Decisions -> Results -> Learnings